

```
    if (!v.empty())
        return {v.back(), v.back().size()}; // 列表初始化
    else
        return pair<string, int>(); // 隐式构造返回值
}
```

若 `v` 不为空，我们返回一个由 `v` 中最后一个 `string` 及其大小组成的 `pair`。否则，隐式构造一个空 `pair`，并返回它。

在较早的 C++ 版本中，不允许用花括号包围的初始化器来返回 `pair` 这种类型的对象，必须显式构造返回值：

```
    if (!v.empty())
        return pair<string, int>(v.back(), v.back().size());
```

我们还可以用 `make_pair` 来生成 `pair` 对象，`pair` 的两个类型来自于 `make_pair` 的参数：

```
    if (!v.empty())
        return make_pair(v.back(), v.back().size());
```

11.2.3 节练习

练习 11.12: 编写程序，读入 `string` 和 `int` 的序列，将每个 `string` 和 `int` 存入一个 `pair` 中，`pair` 保存在一个 `vector` 中。

练习 11.13: 在上一题的程序中，至少有三种创建 `pair` 的方法。编写此程序的三个版本，分别采用不同的方法创建 `pair`。解释你认为哪种形式最易于编写和理解，为什么？

练习 11.14: 扩展你在 11.2.1 节练习（第 378 页）中编写的孩子姓到名的 `map`，添加一个 `pair` 的 `vector`，保存孩子的名和生日。

11.3 关联容器操作

除了表 9.2（第 295 页）中列出的类型，关联容器还定义了表 11.3 中列出的类型。这些类型表示容器关键字和值的类型。

表 11.3: 关联容器额外的类型别名	
<code>key_type</code>	此容器类型的关键字类型
<code>mapped_type</code>	每个关键字关联的类型；只适用于 <code>map</code>
<code>value_type</code>	对于 <code>set</code> ，与 <code>key_type</code> 相同 对于 <code>map</code> ，为 <code>pair<const key_type, mapped_type></code>

对于 `set` 类型，**`key_type`** 和 **`value_type`** 是一样的；`set` 中保存的值就是关键字。在一个 `map` 中，元素是关键字-值对。即，每个元素是一个 `pair` 对象，包含一个关键字和一个关联的值。由于我们不能改变一个元素的关键字，因此这些 `pair` 的关键字部分是 `const` 的：

```
set<string>::value_type v1; // v1 是一个 string
set<string>::key_type v2;   // v2 是一个 string
map<string, int>::value_type v3; // v3 是一个 pair<const string, int>
map<string, int>::key_type v4; // v4 是一个 string
map<string, int>::mapped_type v5; // v5 是一个 int
```

与顺序容器一样（参见 9.2.2 节，第 297 页），我们使用作用域运算符来提取一个类型的成员——例如，`map<string, int>::key_type`。

只有 `map` 类型（`unordered_map`、`unordered_multimap`、`multimap` 和 `map`）才定义了 `mapped_type`。

11.3.1 关联容器迭代器

当解引用一个关联容器迭代器时，我们会得到一个类型为容器的 `value_type` 的值的引用。对 `map` 而言，`value_type` 是一个 `pair` 类型，其 `first` 成员保存 `const` 的关键字，`second` 成员保存值：

```
// 获得指向 word_count 中一个元素的迭代器
auto map_it = word_count.begin();
// *map_it 是指向一个 pair<const string, size_t> 对象的引用
cout << map_it->first;           // 打印此元素的关键字
cout << " " << map_it->second;   // 打印此元素的值
map_it->first = "new key";       // 错误：关键字是 const 的
++map_it->second; // 正确：我们可以通过迭代器改变元素
```



必须记住，一个 `map` 的 `value_type` 是一个 `pair`，我们可以改变 `pair` 的值，但不能改变关键字成员的值。

set 的迭代器是 const 的

虽然 `set` 类型同时定义了 `iterator` 和 `const_iterator` 类型，但两种类型都只允许只读访问 `set` 中的元素。与不能改变一个 `map` 元素的关键字一样，一个 `set` 中的关键字也是 `const` 的。可以用一个 `set` 迭代器来读取元素的值，但不能修改：

```
set<int> iset = {0,1,2,3,4,5,6,7,8,9};
set<int>::iterator set_it = iset.begin();
if (set_it != iset.end()) {
    *set_it = 42;           // 错误：set 中的关键字是只读的
    cout << *set_it << endl; // 正确：可以读关键字
}
```

430

遍历关联容器

`map` 和 `set` 类型都支持表 9.2（第 295 页）中的 `begin` 和 `end` 操作。与往常一样，我们可以用这些函数获取迭代器，然后用迭代器来遍历容器。例如，我们可以编写一个循环来打印第 375 页中单词计数程序的结果，如下所示：

```
// 获得一个指向首元素的迭代器
auto map_it = word_count.cbegin();
// 比较当前迭代器和尾后迭代器
while (map_it != word_count.cend()) {
    // 解引用迭代器，打印关键字-值对
    cout << map_it->first << " occurs "
         << map_it->second << " times" << endl;
    ++map_it; // 递增迭代器，移动到下一个元素
}
```

`while` 的循环条件和循环中的迭代器递增操作看起来很像我们之前编写的打印一个 `vector`

或一个 `string` 的程序。我们首先初始化迭代器 `map_it`，让它指向 `word_count` 中的首元素。只要迭代器不等于 `end`，就打印当前元素并递增迭代器。输出语句解引用 `map_it` 来获得 `pair` 的成员，否则与我们之前的程序一样。



本程序的输出是按字典序排列的。当使用一个迭代器遍历一个 `map`、`multimap`、`set` 或 `multiset` 时，迭代器按关键字升序遍历元素。

关联容器和算法

我们通常不对关联容器使用泛型算法（参见第 10 章）。关键字是 `const` 这一特性意味着不能将关联容器传递给修改或重排容器元素的算法，因为这类算法需要向元素写入值，而 `set` 类型中的元素是 `const` 的，`map` 中的元素是 `pair`，其第一个成员是 `const` 的。

关联容器可用于只读取元素的算法。但是，很多这类算法都要搜索序列。由于关联容器中的元素不能通过它们的关键字进行（快速）查找，因此对其使用泛型搜索算法几乎总是个坏主意。例如，我们将在 11.3.5 节（第 388 页）中看到，关联容器定义了一个名为 `find` 的成员，它通过一个给定的关键字直接获取元素。我们可以用泛型 `find` 算法来查找一个元素，但此算法会进行顺序搜索。使用关联容器定义的专用的 `find` 成员会比调用泛型 `find` 快得多。

在实际编程中，如果我们真要对一个关联容器使用算法，要么是将它当作一个源序列，要么当作一个目的位置。例如，可以用泛型 `copy` 算法将元素从一个关联容器拷贝到另一个序列。类似的，可以调用 `inserter` 将一个插入器绑定（参见 10.4.1 节，第 358 页）到一个关联容器。通过使用 `inserter`，我们可以将关联容器当作一个目的位置来调用另一个算法。

11.3.1 节练习

431

练习 11.15: 对一个 `int` 到 `vector<int>` 的 `map`，其 `mapped_type`、`key_type` 和 `value_type` 分别是什么？

练习 11.16: 使用一个 `map` 迭代器编写一个表达式，将一个值赋予一个元素。

练习 11.17: 假定 `c` 是一个 `string` 的 `multiset`，`v` 是一个 `string` 的 `vector`，解释下面的调用。指出每个调用是否合法：

```
copy(v.begin(), v.end(), inserter(c, c.end()));  
copy(v.begin(), v.end(), back_inserter(c));  
copy(c.begin(), c.end(), inserter(v, v.end()));  
copy(c.begin(), c.end(), back_inserter(v));
```

练习 11.18: 写出第 382 页循环中 `map_it` 的类型，不要使用 `auto` 或 `decltype`。

练习 11.19: 定义一个变量，通过对 11.2.2 节（第 378 页）中的名为 `bookstore` 的 `multiset` 调用 `begin()` 来初始化这个变量。写出变量的类型，不要使用 `auto` 或 `decltype`。

11.3.2 添加元素

关联容器的 `insert` 成员（见表 11.4，第 384 页）向容器中添加一个元素或一个元素范围。由于 `map` 和 `set`（以及对应的无序类型）包含不重复的关键字，因此插入一个已

存在的元素对容器没有任何影响：

```
vector<int> ivec = {2,4,6,8,2,4,6,8};           // ivec 有 8 个元素
set<int> set2;                                   // 空集合
set2.insert(ivec.cbegin(), ivec.cend());         // set2 有 4 个元素
set2.insert({1,3,5,7,1,3,5,7});                 // set2 现在有 8 个元素
```

insert 有两个版本，分别接受一对迭代器，或是一个初始化器列表，这两个版本的行为类似对应的构造函数（参见 11.2.1 节，第 376 页）——对于一个给定的关键字，只有第一个带此关键字的元素才被插入到容器中。

向 map 添加元素

对一个 map 进行 insert 操作时，必须记住元素类型是 pair。通常，对于想要插入的数据，并没有一个现成的 pair 对象。可以在 insert 的参数列表中创建一个 pair：

```
// 向 word_count 插入 word 的 4 种方法
word_count.insert({word, 1});
word_count.insert(make_pair(word, 1));
word_count.insert(pair<string, size_t>(word, 1));
word_count.insert(map<string, size_t>::value_type(word, 1));
```

C++
11

如我们所见，在新标准下，创建一个 pair 最简单的方法是在参数列表中使用花括号初始化。也可以调用 make_pair 或显式构造 pair。最后一个 insert 调用中的参数：

432

```
map<string, size_t>::value_type(s, 1)
```

构造一个恰当的 pair 类型，并构造该类型的一个新对象，插入到 map 中。

表 11.4: 关联容器 insert 操作

c.insert(v)	v 是 value_type 类型的对象；args 用来构造一个元素
c.emplace(args)	对于 map 和 set，只有当元素的关键字不在 c 中时才插入（或构造）元素。函数返回一个 pair，包含一个迭代器，指向具有指定关键字的元素，以及一个指示插入是否成功的 bool 值。 对于 multimap 和 multiset，总会插入（或构造）给定元素，并返回一个指向新元素的迭代器
c.insert(b, e)	b 和 e 是迭代器，表示一个 c::value_type 类型值的范围；il 是这种值的花括号列表。函数返回 void
c.insert(il)	对于 map 和 set，只插入关键字不在 c 中的元素。对于 multimap 和 multiset，则会插入范围中的每个元素
c.insert(p, v)	类似 insert(v)（或 emplace(args)），但将迭代器 p 作为一个提示，指出从哪里开始搜索新元素应该存储的位置。返回一个迭代器，指向具有给定关键字的元素
c.emplace(p, args)	

检测 insert 的返回值

insert（或 emplace）返回的值依赖于容器类型和参数。对于不包含重复关键字的容器，添加单一元素的 insert 和 emplace 版本返回一个 pair，告诉我们插入操作是否成功。pair 的 first 成员是一个迭代器，指向具有给定关键字的元素；second 成员是一个 bool 值，指出元素是插入成功还是已经存在于容器中。如果关键字已在容器中，则 insert 什么事情也不做，且返回值中的 bool 部分为 false。如果关键字不存在，元

素被插入容器中，且 `bool` 值为 `true`。

作为一个例子，我们用 `insert` 重写单词计数程序：

```
// 统计每个单词在输入中出现次数的一种更烦琐的方法
map<string, size_t> word_count; // 从 string 到 size_t 的空 map
string word;
while (cin >> word) {
    // 插入一个元素，关键字等于 word，值为 1；
    // 若 word 已在 word_count 中，insert 什么也不做
    auto ret = word_count.insert({word, 1});
    if (!ret.second) // word 已在 word_count 中
        ++ret.first->second; // 递增计数器
}
```

对于每个 `word`，我们尝试将其插入到容器中，对应的值为 1。若 `word` 已在 `map` 中，则什么都不做，特别是与 `word` 相关联的计数器的值不变。若 `word` 还未在 `map` 中，则此 `string` 对象被添加到 `map` 中，且其计数器的值被置为 1。

◀ 433

`if` 语句检查返回值的 `bool` 部分，若为 `false`，则表明插入操作未发生。在此情况下，`word` 已存在于 `word_count` 中，因此必须递增此元素所关联的计数器。

展开递增语句

在这个版本的单词计数程序中，递增计数器的语句很难理解。通过添加一些括号来反映出运算符的优先级（参见 4.1.2 节，第 121 页），会使表达式更容易理解一些：

```
++((ret.first)->second); // 等价的表达式
```

下面我们一步一步来解释此表达式：

`ret` 保存 `insert` 返回的值，是一个 `pair`。

`ret.first` 是 `pair` 的第一个成员，是一个 `map` 迭代器，指向具有给定关键字的元素。

`ret.first->` 解引用此迭代器，提取 `map` 中的元素，元素也是一个 `pair`。

`ret.first->second` `map` 中元素的值部分。

`++ret.first->second` 递增此值。

再回到原来完整的递增语句，它提取匹配关键字 `word` 的元素的迭代器，并递增与我们试图插入的关键字相关联的计数器。

如果读者使用的是旧版本的编译器，或者是在阅读新标准推出之前编写的代码，`ret` 的声明和初始化可能复杂些：

```
pair<map<string, size_t>::iterator, bool> ret =
    word_count.insert(make_pair(word, 1));
```

应该容易看出这条语句定义了一个 `pair`，其第二个类型为 `bool` 类型。第一个类型理解起来有点儿困难，它是一个在 `map<string, size_t>` 类型上定义的 `iterator` 类型。

向 `multiset` 或 `multimap` 添加元素

我们的单词计数程序依赖于这样一个事实：一个给定的关键字只能出现一次。这样，任意给定的单词只有一个关联的计数器。我们有时希望能添加具有相同关键字的多个元素。例如，可能想建立作者到他所著书籍题目的映射。在此情况下，每个作者可能有多个

条目，因此我们应该使用 `multimap` 而不是 `map`。由于一个 `multi` 容器中的关键字不必唯一，在这些类型上调用 `insert` 总会插入一个元素：

434

```
multimap<string, string> authors;
// 插入第一个元素，关键字为 Barth, John
authors.insert({"Barth, John", "Sot-Weed Factor"});
// 正确：添加第二个元素，关键字也是 Barth, John
authors.insert({"Barth, John", "Lost in the Funhouse"});
```

对允许重复关键字的容器，接受单个元素的 `insert` 操作返回一个指向新元素的迭代器。这里无须返回一个 `bool` 值，因为 `insert` 总是向这类容器中加入一个新元素。

11.3.2 节练习

练习 11.20: 重写 11.1 节练习（第 376 页）的单词计数程序，使用 `insert` 代替下标操作。你认为哪个程序更容易编写和阅读？解释原因。

练习 11.21: 假定 `word_count` 是一个 `string` 到 `size_t` 的 `map`，`word` 是一个 `string`，解释下面循环的作用：

```
while (cin >> word)
    ++word_count.insert({word, 0}).first->second;
```

练习 11.22: 给定一个 `map<string, vector<int>>`，对此容器的插入一个元素的 `insert` 版本，写出其参数类型和返回类型。

练习 11.23: 11.2.1 节练习（第 378 页）中的 `map` 以孩子的姓为关键字，保存他们的名的 `vector`，用 `multimap` 重写此 `map`。

11.3.3 删除元素

关联容器定义了三个版本的 `erase`，如表 11.5 所示。与顺序容器一样，我们可以通过传递给 `erase` 一个迭代器或一个迭代器对来删除一个元素或者一个元素范围。这两个版本的 `erase` 与对应的顺序容器的操作非常相似：指定的元素被删除，函数返回 `void`。

关联容器提供一个额外的 `erase` 操作，它接受一个 `key_type` 参数。此版本删除所有匹配给定关键字的元素（如果存在的话），返回实际删除的元素的数量。我们可以用此版本在打印结果之前从 `word_count` 中删除一个特定的单词：

```
// 删除一个关键字，返回删除的元素数量
if (word_count.erase(removal_word))
    cout << "ok: " << removal_word << " removed\n";
else cout << "oops: " << removal_word << " not found!\n";
```

对于保存不重复关键字的容器，`erase` 的返回值总是 0 或 1。若返回值为 0，则表明想要删除的元素并不在容器中

435

对允许重复关键字的容器，删除元素的数量可能大于 1：

```
auto cnt = authors.erase("Barth, John");
```

如果 `authors` 是我们在 11.3.2 节（第 386 页）中创建的 `multimap`，则 `cnt` 的值为 2。

表 11.5: 从关联容器删除元素	
<code>c.erase(k)</code>	从 <code>c</code> 中删除每个关键字为 <code>k</code> 的元素。返回一个 <code>size_type</code> 值，指出删除的元素的数量
<code>c.erase(p)</code>	从 <code>c</code> 中删除迭代器 <code>p</code> 指定的元素。 <code>p</code> 必须指向 <code>c</code> 中一个真实元素，不能等于 <code>c.end()</code> 。返回一个指向 <code>p</code> 之后元素的迭代器，若 <code>p</code> 指向 <code>c</code> 中的尾元素，则返回 <code>c.end()</code>
<code>c.erase(b, e)</code>	删除迭代器对 <code>b</code> 和 <code>e</code> 所表示的范围中的元素。返回 <code>e</code>

11.3.4 map 的下标操作



`map` 和 `unordered_map` 容器提供了下标运算符和一个对应的 `at` 函数（参见 9.3.2 节，第 311 页），如表 11.6 所示。`set` 类型不支持下标，因为 `set` 中没有与关键字相关联的“值”。元素本身就是关键字，因此“获取与一个关键字相关联的值”的操作就没有意义了。我们不能对一个 `multimap` 或一个 `unordered_multimap` 进行下标操作，因为这些容器中可能有多个值与一个关键字相关联。

类似我们用过的其他下标运算符，`map` 下标运算符接受一个索引（即，一个关键字），获取与此关键字相关联的值。但是，与其他下标运算符不同的是，如果关键字并不在 `map` 中，会为它创建一个元素并插入到 `map` 中，关联值将进行值初始化（参见 3.3.1 节，第 88 页）。

例如，如果我们编写如下代码

```
map<string, size_t> word_count; // empty map
// 插入一个关键字为 Anna 的元素，关联值进行值初始化；然后将 1 赋予它
word_count["Anna"] = 1;
```

将会执行如下操作：

- 在 `word_count` 中搜索关键字为 `Anna` 的元素，未找到。
- 将一个新的关键字-值对插入到 `word_count` 中。关键字是一个 `const string`，保存 `Anna`。值进行值初始化，在本例中意味着值为 0。
- 提取出新插入的元素，并将值 1 赋予它。

由于下标运算符可能插入一个新元素，我们只可以对非 `const` 的 `map` 使用下标操作。



对一个 `map` 使用下标操作，其行为与数组或 `vector` 上的下标操作很不相同：使用一个不在容器中的关键字作为下标，会添加一个具有此关键字的元素到 `map` 中。

表 11.6: map 和 unordered_map 的下标操作	
<code>c[k]</code>	返回关键字为 <code>k</code> 的元素；如果 <code>k</code> 不在 <code>c</code> 中，添加一个关键字为 <code>k</code> 的元素，对其进行值初始化
<code>c.at(k)</code>	访问关键字为 <code>k</code> 的元素，带参数检查；若 <code>k</code> 不在 <code>c</code> 中，抛出一个 <code>out_of_range</code> 异常（参见 5.6 节，第 173 页）

使用下标操作的返回值

`map` 的下标运算符与我们用过的其他下标运算符的另一个不同之处是其返回类型。通

常情况下，解引用一个迭代器所返回的类型与下标运算符返回的类型是一样的。但对 map 则不然：当对一个 map 进行下标操作时，会获得一个 mapped_type 对象；但当解引用一个 map 迭代器时，会得到一个 value_type 对象（参见 11.3 节，第 381 页）。

与其他下标运算符相同的是，map 的下标运算符返回一个左值（参见 4.1.1 节，第 121 页）。由于返回的是一个左值，所以我们既可以读也可以写元素：

```
cout << word_count["Anna"]; // 用 Anna 作为下标提取元素；会打印出 1
++word_count["Anna"];      // 提取元素，将其增 1
cout << word_count["Anna"]; // 提取元素并打印它；会打印出 2
```

与 vector 与 string 不同，map 的下标运算符返回的类型与解引用 map 迭代器得到的类型不同。

如果关键字还未在 map 中，下标运算符会添加一个新元素，这一特性允许我们编写出异常简洁的程序，例如单词计数程序中的循环（参见 11.1 节，第 375 页）。另一方面，有时只是想知道一个元素是否已在 map 中，但在不存在时并不想添加元素。在这种情况下，就不能使用下标运算符。

11.3.4 节练习

练习 11.24: 下面的程序完成什么功能？

```
map<int, int> m;
m[0] = 1;
```

练习 11.25: 对比下面程序与上一题程序

```
vector<int> v;
v[0] = 1;
```

练习 11.26: 可以用什么类型来对一个 map 进行下标操作？下标运算符返回的类型是什么？请给出一个具体例子——即，定义一个 map，然后写出一个可以用来对 map 进行下标操作的类型以及下标运算符将会返回的类型。

11.3.5 访问元素

关联容器提供多种查找一个指定元素的方法，如表 11.7 所示。应该使用哪个操作依赖于我们要解决什么问题。如果我们所关心的只不过是一个特定元素是否已在容器中，可能 find 是最佳选择。对于不允许重复关键字的容器，可能使用 find 还是 count 没什么区别。但对于允许重复关键字的容器，count 还会做更多的工作：如果元素在容器中，它还会统计有多少个元素有相同的关键字。如果不需要计数，最好使用 find：

437

```
set<int> iset = {0,1,2,3,4,5,6,7,8,9};
iset.find(1); // 返回一个迭代器，指向 key == 1 的元素
iset.find(11); // 返回一个迭代器，其值等于 iset.end()
iset.count(1); // 返回 1
iset.count(11); // 返回 0
```

表 11.7: 在一个关联容器中查找元素的操作
lower_bound 和 upper_bound 不适用于无序容器。
下标和 at 操作只适用于非 const 的 map 和 unordered_map。

续表

<code>c.find(k)</code>	返回一个迭代器，指向第一个关键字为 <code>k</code> 的元素，若 <code>k</code> 不在容器中，则返回尾后迭代器
<code>c.count(k)</code>	返回关键字等于 <code>k</code> 的元素的数量。对于不允许重复关键字的容器，返回值永远是 0 或 1
<code>c.lower_bound(k)</code>	返回一个迭代器，指向第一个关键字不小于 <code>k</code> 的元素
<code>c.upper_bound(k)</code>	返回一个迭代器，指向第一个关键字大于 <code>k</code> 的元素
<code>c.equal_range(k)</code>	返回一个迭代器 <code>pair</code> ，表示关键字等于 <code>k</code> 的元素的范围。若 <code>k</code> 不存在， <code>pair</code> 的两个成员均等于 <code>c.end()</code>

对 map 使用 find 代替下标操作

对 `map` 和 `unordered_map` 类型，下标运算符提供了最简单的提取元素的方法。但是，如我们所见，使用下标操作有一个严重的副作用：如果关键字还未在 `map` 中，下标操作会插入一个具有给定关键字的元素。这种行为是否正确完全依赖于我们的预期是什么。例如，单词计数程序依赖于这样一个特性：使用一个不存在的关键字作为下标，会插入一个新元素，其关键字为给定关键字，其值为 0。也就是说，下标操作的行为符合我们的预期。

但有时，我们只是想知道一个给定关键字是否在 `map` 中，而不想改变 `map`。这样就不能使用下标运算符来检查一个元素是否存在，因为如果关键字不存在的话，下标运算符会插入一个新元素。在这种情况下，应该使用 `find`：

```
if (word_count.find("foobar") == word_count.end())
    cout << "foobar is not in the map" << endl;
```

在 multimap 或 multiset 中查找元素

在一个不允许重复关键字的关联容器中查找一个元素是一件很简单的事情——元素要么在容器中，要么不在。但对于允许重复关键字的容器来说，过程就更为复杂：在容器中可能有很多元素具有给定的关键字。如果一个 `multimap` 或 `multiset` 中有多个元素具有给定关键字，则这些元素在容器中会相邻存储。

例如，给定一个从作者到著作题目的映射，我们可能想打印一个特定作者的所有著作。可以用三种不同方法来解决这个问题。最直观的方法是使用 `find` 和 `count`：

438

```
string search_item("Alain de Botton");           // 要查找的作者
auto entries = authors.count(search_item);       // 元素的数量
auto iter = authors.find(search_item);           // 此作者的第一本书
// 用一个循环查找此作者的所有著作
while(entries) {
    cout << iter->second << endl;                 // 打印每个题目
    ++iter;                                       // 前进到下一本书
    --entries;                                   // 记录已经打印了多少本书
}
```

首先调用 `count` 确定此作者共有多少本著作，并调用 `find` 获得一个迭代器，指向第一个关键字为此作者的元素。for 循环的迭代次数依赖于 `count` 的返回值。特别是，如果 `count` 返回 0，则循环一次也不执行。



当我们遍历一个 `multimap` 或 `multiset` 时，保证可以得到序列中所有具有给定关键字的元素。

一种不同的，面向迭代器的解决方法

我们还可以用 `lower_bound` 和 `upper_bound` 来解决此问题。这两个操作都接受一个关键字，返回一个迭代器。如果关键字在容器中，`lower_bound` 返回的迭代器将指向第一个具有给定关键字的元素，而 `upper_bound` 返回的迭代器则指向最后一个匹配给定关键字的元素之后的位置。如果元素不在 `multimap` 中，则 `lower_bound` 和 `upper_bound` 会返回相等的迭代器——指向一个不影响排序的关键字插入位置。因此，用相同的关键字调用 `lower_bound` 和 `upper_bound` 会得到一个迭代器范围（参见 9.2.1 节，第 296 页），表示所有具有该关键字的元素的范围。

439

当然，这两个操作返回的迭代器可能是容器的尾后迭代器。如果我们查找的元素具有容器中最大的关键字，则此关键字的 `upper_bound` 返回尾后迭代器。如果关键字不存在，且大于容器中任何关键字，则 `lower_bound` 返回的也是尾后迭代器。



`lower_bound` 返回的迭代器可能指向一个具有给定关键字的元素，但也可能不指向。如果关键字不在容器中，则 `lower_bound` 会返回关键字的第一个安全插入点——不影响容器中元素顺序的插入位置。

使用这两个操作，我们可以重写前面的程序：

```
// authors 和 search_item 的定义，与前面的程序一样
// beg 和 end 表示对应此作者的元素的范围
for (auto beg = authors.lower_bound(search_item),
     end = authors.upper_bound(search_item);
     beg != end; ++beg)
    cout << beg->second << endl; // 打印每个题目
```

此程序与使用 `count` 和 `find` 的版本完成相同的工作，但更直接。对 `lower_bound` 的调用将 `beg` 定位到第一个与 `search_item` 匹配的元素（如果存在的话）。如果容器中没有这样的元素，`beg` 将指向第一个关键字大于 `search_item` 的元素，有可能是尾后迭代器。`upper_bound` 调用将 `end` 指向最后一个匹配指定关键字的元素之后的元素。这两个操作并不报告关键字是否存在，重要的是它们的返回值可作为一个迭代器范围（参见 9.2.1 节，第 296 页）。

如果没有元素与给定关键字匹配，则 `lower_bound` 和 `upper_bound` 会返回相等的迭代器——都指向给定关键字的插入点，能保持容器中元素顺序的插入位置。

假定有多个元素与给定关键字匹配，`beg` 将指向其中第一个元素。我们可以通过递增 `beg` 来遍历这些元素。`end` 中的迭代器会指出何时完成遍历——当 `beg` 等于 `end` 时，就表明已经遍历了所有匹配给定关键字的元素了。

由于这两个迭代器构成一个范围，我们可以用一个 `for` 循环来遍历这个范围。循环可能执行零次，如果存在给定作者的话，就会执行多次，打印出该作者的所有项。如果给定作者不存在，`beg` 和 `end` 相等，循环就一次也不会执行。否则，我们知道递增 `beg` 最终会使它到达 `end`，在此过程中我们就会打印出与此作者关联的每条记录。



如果 `lower_bound` 和 `upper_bound` 返回相同的迭代器，则给定关键字不在容器中。

`equal_range` 函数

解决此问题的最后一种方法是三种方法中最直接的：不必再调用 `upper_bound` 和

lower_bound, 直接调用 equal_range 即可。此函数接受一个关键字, 返回一个迭代器 pair。若关键字存在, 则第一个迭代器指向第一个与关键字匹配的元素, 第二个迭代器指向最后一个匹配元素之后的位置。若未找到匹配元素, 则两个迭代器都指向关键字可以插入的位置。

440

可以用 equal_range 来再次修改我们的程序:

```
// authors 和 search_item 的定义, 与前面的程序一样
// pos 保存迭代器对, 表示与关键字匹配的元素范围
for (auto pos = authors.equal_range(search_item);
     pos.first != pos.second; ++pos.first)
    cout << pos.first->second << endl; // 打印每个题目
```

此程序本质上与前一个使用 upper_bound 和 lower_bound 的程序是一样的。不同之处就是, 没有用局部变量 beg 和 end 来保存元素范围, 而是使用了 equal_range 返回的 pair。此 pair 的 first 成员保存的迭代器与 lower_bound 返回的迭代器是一样的, second 保存的迭代器与 upper_bound 的返回值是一样的。因此, 在此程序中, pos.first 等价于 beg, pos.second 等价于 end。

11.3.5 节练习

练习 11.27: 对于什么问题你会使用 count 来解决? 什么时候你又会选择 find 呢?

练习 11.28: 对一个 string 到 int 的 vector 的 map, 定义并初始化一个变量来保存在其上调用 find 所返回的结果。

练习 11.29: 如果给定的关键字不在容器中, upper_bound、lower_bound 和 equal_range 分别会返回什么?

练习 11.30: 对于本节最后一个程序中的输出表达式, 解释运算对象 pos.first->second 的含义。

练习 11.31: 编写程序, 定义一个作者及其作品的 multimap。使用 find 在 multimap 中查找一个元素并用 erase 删除它。确保你的程序在元素不在 map 中时也能正常运行。

练习 11.32: 使用上一题定义的 multimap 编写一个程序, 按字典序打印作者列表和他们的作品。

11.3.6 一个单词转换的 map

我们将以一个程序结束本节的内容, 它将展示 map 的创建、搜索以及遍历。这个程序的功能是这样的: 给定一个 string, 将它转换为另一个 string。程序的输入是两个文件。第一个文件保存的是一些规则, 用来转换第二个文件中的文本。每条规则由两部分组成: 一个可能出现在输入文件中的单词和一个用来替换它的短语。表达的含义是, 每当第一个单词出现在输入中时, 我们就将它替换为对应的短语。第二个输入文件包含要转换的文本。

如果单词转换文件的内容如下所示:

```
brb be right back
k okay?
y why
r are
```

441

```

u you
pic picture
thk thanks!
l8r later

```

我们希望转换的文本为

```

where r u
y dont u send me a pic
k thk l8r

```

则程序应该生成这样的输出:

```

where are you
why dont you send me a picture
okay? thanks! later

```

单词转换程序

我们的程序将使用三个函数。函数 `word_transform` 管理整个过程。它接受两个 `ifstream` 参数: 第一个参数应绑定到单词转换文件, 第二个参数应绑定到我们要转换的文本文件。函数 `buildMap` 会读取转换规则文件, 并创建一个 `map`, 用于保存每个单词到其转换内容的映射。函数 `transform` 接受一个 `string`, 如果存在转换规则, 返回转换后的内容。

我们首先定义 `word_transform` 函数。最重要的部分是调用 `buildMap` 和 `transform`:

```

void word_transform(ifstream &map_file, ifstream &input)
{
    auto trans_map = buildMap(map_file); // 保存转换规则
    string text;                          // 保存输入中的每一行
    while (getline(input, text)) {         // 读取一行输入
        istringstream stream(text);       // 读取每个单词
        string word;
        bool firstword = true;            // 控制是否打印空格
        while (stream >> word) {
            if (firstword)
                firstword = false;
            else
                cout << " ";              // 在单词间打印一个空格
            // transform 返回它的第一个参数或其转换之后的形式
            cout << transform(word, trans_map); // 打印输出
        }
        cout << endl;                    // 完成一行的转换
    }
}

```

442 函数首先调用 `buildMap` 来生成单词转换 `map`, 我们将它保存在 `trans_map` 中。函数的剩余部分处理输入文件。`while` 循环用 `getline` 一行一行地读取输入文件。这样做的目的是使得输出中的换行位置能和输入文件中一样。为了从每行中获取单词, 我们使用了一个嵌套的 `while` 循环, 它用一个 `istringstream` (参见 8.3 节, 第 287 页) 来处理当前行中的每个单词。

在输出过程中, 内层 `while` 循环使用一个 `bool` 变量 `firstword` 来确定是否打印

一个空格。它通过调用 `transform` 来获得要打印的单词。`transform` 的返回值或者是 `word` 中原来的 `string`，或者是 `trans_map` 中指出的对应的转换内容。

建立转换映射

函数 `buildMap` 读入给定文件，建立起转换映射。

```
map<string, string> buildMap(istream &map_file)
{
    map<string, string> trans_map; // 保存转换规则
    string key;                  // 要转换的单词
    string value;                // 替换后的内容
    // 读取第一个单词存入 key 中，行中剩余内容存入 value
    while (map_file >> key && getline(map_file, value))
        if (value.size() > 1) // 检查是否有转换规则
            trans_map[key] = value.substr(1); // 跳过前导空格
        else
            throw runtime_error("no rule for " + key);
    return trans_map;
}
```

`map_file` 中的每一行对应一条规则。每条规则由一个单词和一个短语组成，短语可能包含多个单词。我们用 `>>` 读取要转换的单词，存入 `key` 中，并调用 `getline` 读取这一行中的剩余内容存入 `value`。由于 `getline` 不会跳过前导空格（参见 3.2.2 节，第 78 页），需要我们来跳过单词和它的转换内容之间的空格。在保存转换规则之前，检查是否获得了一个以上的字符。如果是，调用 `substr`（参见 9.5.1 节，第 321 页）来跳过分隔单词及其转换短语之间的前导空格，并将得到的子字符串存入 `trans_map`。

注意，我们使用下标运算符来添加关键字-值对。我们隐含地忽略了一个单词在转换文件中出现多次的情况。如果真的有单词出现多次，循环会将最后一个对应短语存入 `trans_map`。当 `while` 循环结束后，`trans_map` 中将保存着用来转换输入文本的规则。

生成转换文本

函数 `transform` 进行实际的转换工作。其参数是需要转换的 `string` 的引用和转换规则 `map`。如果给定 `string` 在 `map` 中，`transform` 返回相应的短语。否则，`transform` 直接返回原 `string`：

```
const string &
transform(const string &s, const map<string, string> &m)
{
    // 实际的转换工作；此部分是程序的核心
    auto map_it = m.find(s);
    // 如果单词在转换规则 map 中
    if (map_it != m.cend())
        return map_it->second; // 使用替换短语
    else
        return s;              // 否则返回原 string
}
```

◀ 443

函数首先调用 `find` 来确定给定 `string` 是否在 `map` 中。如果存在，则 `find` 返回一个指向对应元素的迭代器。否则，`find` 返回尾后迭代器。如果元素存在，我们解引用迭代器，获得一个保存关键字和值的 `pair`（参见 11.3 节，第 381 页），然后返回成员 `second`，即

用来替代 `s` 的内容。

11.3.6 节练习

练习 11.33: 实现你自己版本的单词转换程序。

练习 11.34: 如果你将 `transform` 函数中的 `find` 替换为下标运算符, 会发生什么情况?

练习 11.35: 在 `buildMap` 中, 如果进行如下改写, 会有什么效果?

```
trans_map[key] = value.substr(1);
改为 trans_map.insert({key, value.substr(1)})
```

练习 11.36: 我们的程序并没有检查输入文件的合法性。特别是, 它假定转换规则文件中的规则都是有意义的。如果文件中的某一行包含一个关键字、一个空格, 然后就结束了, 会发生什么? 预测程序的行为并进行验证, 再与你的程序进行比较。



11.4 无序容器

新标准定义了 4 个无序关联容器 (unordered associative container)。这些容器不是使用比较运算符来组织元素, 而是使用一个哈希函数 (hash function) 和关键字类型的 `==` 运算符。在关键字类型的元素没有明显的序关系的情况下, 无序容器是非常有用的。在某些应用中, 维护元素的序代价非常高昂, 此时无序容器也很有用。

虽然理论上哈希技术能获得更好的平均性能, 但在实际中想要达到很好的效果还需要进行一些性能测试和调优工作。因此, 使用无序容器通常更为简单 (通常也会有更好的性能)。

444



如果关键字类型固有就是无序的, 或者性能测试发现问题可以用哈希技术解决, 就可以使用无序容器。

使用无序容器

除了哈希管理操作之外, 无序容器还提供了与有序容器相同的操作 (`find`、`insert` 等)。这意味着我们曾用于 `map` 和 `set` 的操作也能用于 `unordered_map` 和 `unordered_set`。类似的, 无序容器也有允许重复关键字的版本。

因此, 通常可以用一个无序容器替换对应的有序容器, 反之亦然。但是, 由于元素未按顺序存储, 一个使用无序容器的程序的输出 (通常) 会与使用有序容器的版本不同。

例如, 可以用 `unordered_map` 重写最初的单词计数程序 (参见 11.1 节, 第 375 页):

```
// 统计出现次数, 但单词不会按字典序排列
unordered_map<string, size_t> word_count;
string word;
while (cin >> word)
    ++word_count[word];           // 提取并递增 word 的计数器
for (const auto &w : word_count) // 对 map 中的每个元素
    // 打印结果
    cout << w.first << " occurs " << w.second
        << ((w.second > 1) ? " times" : " time") << endl;
```